

ФГБОУ ВО «Пензенский государственный университет»

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Динамические списки»

Выполнили студенты группы 25ВВВ3:

Махотин Д.А.

Лысов И.В.

Приняли:

к.т.н, доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Цель: Изучить принцип организации динамических списков и получить практические навыки работы с ними

Ход работы: В ходе выполнения лабораторной работы была разработана программа на языке C, использующая динамический односвязный список. Для хранения данных был создан узел списка, содержащий информационное поле, приоритет элемента и указатель на следующий узел. Память для новых элементов выделяется динамически с помощью malloc, а после удаления элементы освобождаются с помощью free.

В программе были реализованы следующие операции:

- Создание элемента списка — ввод информационной строки и приоритета, выделение памяти и инициализация указателя на следующий элемент.
- Работа с очередью — добавление элемента в конец списка и удаление элемента из его начала.
- Работа со стеком — добавление и удаление элемента в начале списка.
- Работа с приоритетной очередью — вставка элементов в порядке убывания приоритета.
- Просмотр списка — вывод информации обо всех элементах.
- Поиск элемента — поиск узла по заданной строке.
- Удаление элемента — удаление найденного узла с корректным обновлением указателей.
- Очистка списка — освобождение памяти, занятой элементами.

Реализованные операции соответствуют заданиям лабораторной работы по созданию приоритетной очереди, очереди и стека на основе динамического списка.

Создана структура узла динамического списка

```
#include <string.h>
#include <locale.h>
▼ struct node
{
    char inf[256];           // данные объекта
    int priority;           // приоритет объекта
    struct node* next;      // указатель на следующий элемент
};

struct node* head = NULL;
struct node* last = NULL;
[]
// Создание нового элемента
▼ struct node* get_struct(void)
{
```

Разработаны операции очереди и стека

```
// Добавление в начало – стек
void stack_push(void)
{
    struct node* p = get_struct();

    p->next = head;
    head = p;

    if (last == NULL)
        last = p;

    printf("Элемент добавлен в стек\n");
}

// Извлечение из начала – стек
void stack_pop(void)
{
    if (head == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

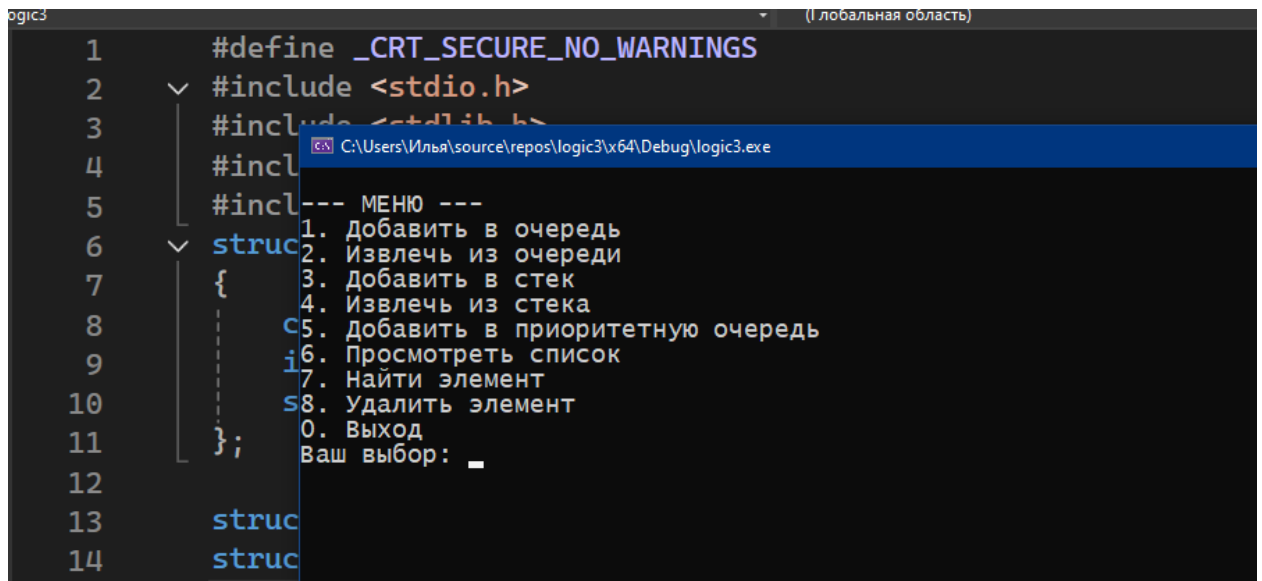
    struct node* p = head;
    head = head->next;

    if (head == NULL)
        last = NULL;

    printf("Извлечен объект: %s\n", p->inf);
    free(p);
}
```

- реализована вставка элементов по убыванию приоритета;
- добавлены функции просмотра, поиска и удаления элементов;
- предусмотрено освобождение выделенной динамической памяти.

Меню программы:

The image shows a screenshot of a C program running in a debugger. The program is named 'logic3' and is running in a global scope. The code is displayed in a dark-themed editor with line numbers 1 through 14. The code includes standard headers and defines a menu. The menu is displayed in a blue box over the code, listing 8 options: 1. Добавить в очередь, 2. Извлечь из очереди, 3. Добавить в стек, 4. Извлечь из стека, 5. Добавить в приоритетную очередь, 6. Просмотреть список, 7. Найти элемент, 8. Удалить элемент, and 0. Выход. The prompt 'Ваш выбор: _' is shown at the bottom of the menu. The code in the background shows the beginning of a struct definition for a queue element.

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <conio.h>
6  struct tnode
7  {
8      int data;
9      struct tnode *next;
10     };
11
12
13 struct tnode *head = NULL;
14 struct tnode *tail = NULL;
```

--- МЕНЮ ---

1. Добавить в очередь
2. Извлечь из очереди
3. Добавить в стек
4. Извлечь из стека
5. Добавить в приоритетную очередь
6. Просмотреть список
7. Найти элемент
8. Удалить элемент
0. Выход

Ваш выбор: _

Вывод: В ходе лабораторной работы были изучены динамические списки и принципы работы с указателями и динамической памятью в языке C.